Generative Reasoning Re-ranker

Mingfu Liang[1], **Yufei Li**[1], **Jay Xu**[1], **Kavosh Asadi**, **Xi Liu**[2], **Shuo Gu**, **Kaushik Rangadurai**, **Frank Shyu**, **Shuaiwen Wang**, **Song Yang**, **Zhijing Li**, **Jiang Liu**, **Mengying Sun**, **Fei Tian**, **Xiaohan Wei**, **Chonglin Sun**, **Jacob Tao**, **Shike Mei**, **Wenlin Chen**, **Hamed Firooz**[3], **Luke Simon**[3]

Meta AI
[1]Joint First Author, [2]Project Lead, [3]Direction Lead

Recent studies increasingly explore Large Language Models (LLMs) as a new paradigm for recommendation systems due to their scalability and world knowledge. However, existing work has three key limitations: (1) most efforts focus on retrieval and ranking, while the reranking phase, critical for refining final recommendations, is largely overlooked; (2) LLMs are typically used in zero-shot or supervised fine-tuning settings, leaving their reasoning abilities, especially those enhanced through reinforcement learning (RL) and high-quality reasoning data, underexploited; (3) items are commonly represented by non-semantic IDs, creating major scalability challenges in industrial systems with billions of identifiers. To address these gaps, we propose the Generative Reasoning Reranker (GR2), an end-to-end framework with a three-stage training pipeline tailored for reranking. First, a pretrained LLM is mid-trained on semantic IDs encoded from non-semantic IDs via a tokenizer achieving $\geq$99% uniqueness. Next, a stronger larger-scale LLM generates high-quality reasoning traces through carefully designed prompting and rejection sampling, which are used for supervised fine-tuning to impart foundational reasoning skills. Finally, we apply Decoupled Clip and Dynamic sAmpling Policy Optimization (DAPO), enabling scalable RL supervision with verifiable rewards designed specifically for reranking. Experiments on two real-world datasets demonstrate GR2's effectiveness: it surpasses the state-of-the-art OneRec-Think by 2.4% in Recall@5 and 1.3% in NDCG@5. Ablations confirm that advanced reasoning traces yield substantial gains across metrics. We further find that RL reward design is crucial in reranking: LLMs tend to exploit reward hacking by preserving item order, motivating conditional verifiable rewards to mitigate this behavior and optimize reranking performance.

Date: January 1st, 2026

Meta

1 Introduction

Recommendation systems have become indispensable across a wide spectrum of online platforms, serving to mitigate information overload by intelligently identifying and presenting items aligned with users' interests and needs (Ramanujam et al., 2025; Behdin et al., 2025; Deng et al., 2025). Over the past decade, deep neural networks have been extensively adopted to model the complex relationships between user feedback and vast arrays of item and user features, typically encoded via large-scale embedding tables (Liang et al., 2025; Luo et al., 2025; Zhang et al., 2024, 2022; Wang et al., 2021; Zhou et al., 2019). Recently, Large Language Models (LLMs) have emerged as a transformative paradigm for recommendation systems (Zhou et al., 2025c; Zhang et al., 2025b; Zhao et al., 2024; Wu et al., 2024), driven by their remarkable capacity for continual performance improvement through model scaling, comprehensive world knowledge, and nuanced contextual understanding. Notable examples include P5 (Geng et al., 2022), which unifies diverse recommendation tasks within a single LLM model, OneRec-Think (Liu et al., 2025), which integrates retrieval and ranking stages by fine-tuning LLMs, and PLUM (He et al., 2025) that adapts pre-trained large language models to deliver scalable and efficient recommendations for YouTube, enhancing both retrieval quality and system performance.

Re-ranking is directly responsible for refining and enhancing recommendation outcome and thus a critical component of modern recommender systems (Gao et al., 2025, 2024; Liu et al.). Despite its significance, the re-ranking stage is frequently neglected in recent LLM-based approaches. Furthermore, these studies often deploy LLMs in zero-shot scenarios or fine-tune them on recommendation datasets without reasoning

trace, thereby underutilizing the models' reasoning capabilities—particularly those that can be enhanced through reinforcement learning (RL) and the incorporation of high-quality reasoning data. Another prevailing limitation is the reliance on non-semantic item identifiers, which poses substantial scalability and adaptability challenges in industrial environments, where the sheer volume of item IDs can lead to an unmanageable expansion of the LLM's vocabulary.

To address the aforementioned limitations, we introduce the Generative Reasoning Re-ranker (GR2), an advanced LLM-based recommendation framework that bridges semantic item representations with world knowledge and sophisticated reasoning to effectively re-rank candidate lists. GR2 is architected around a three-stage training pipeline, as illustrated in Figure 1, meticulously tailored for the demands of re-ranking in large-scale recommendation systems. In the first stage, a pre-trained student LLM (e.g., Qwen3-8B (Team, 2025)) undergoes mid-training on semantic item identifiers inspired by OneRec-Think (Liu et al., 2025). These semantic IDs are derived from non-semantic raw item IDs using a tokenizer designed to achieve at least 99% uniqueness, enabling the model to better capture item semantics, generalize robustly across diverse item spaces, and ensure reliable item distinguishability. The second stage leverages a more powerful teacher LLM (e.g., Qw en3-32B (Team, 2025)) with larger model size to generate high-quality, hierarchical reasoning traces. This is accomplished through prompts carefully crafted for re-ranking problem and rigorous rejection sampling, ensuring that the generated traces are both relevant and informative. These reasoning traces serve as the foundation for supervised fine-tuning, equipping the model with basic reasoning skills and enabling it to interpret and connect item semantics with user preferences. In the final stage, GR2 adapts DAPO (Yu et al., 2025) by re-designing its reward function explicitly for the re-ranking task. The adapted DAPO provides scalable supervision through a custom reward function that aligns with the objectives of re-ranking, further refining the student LLM's reasoning capabilities and enhancing its ability to deliver highly relevant and personalized re-ranked candidate list. The main contributions of the paper are summarized as follows

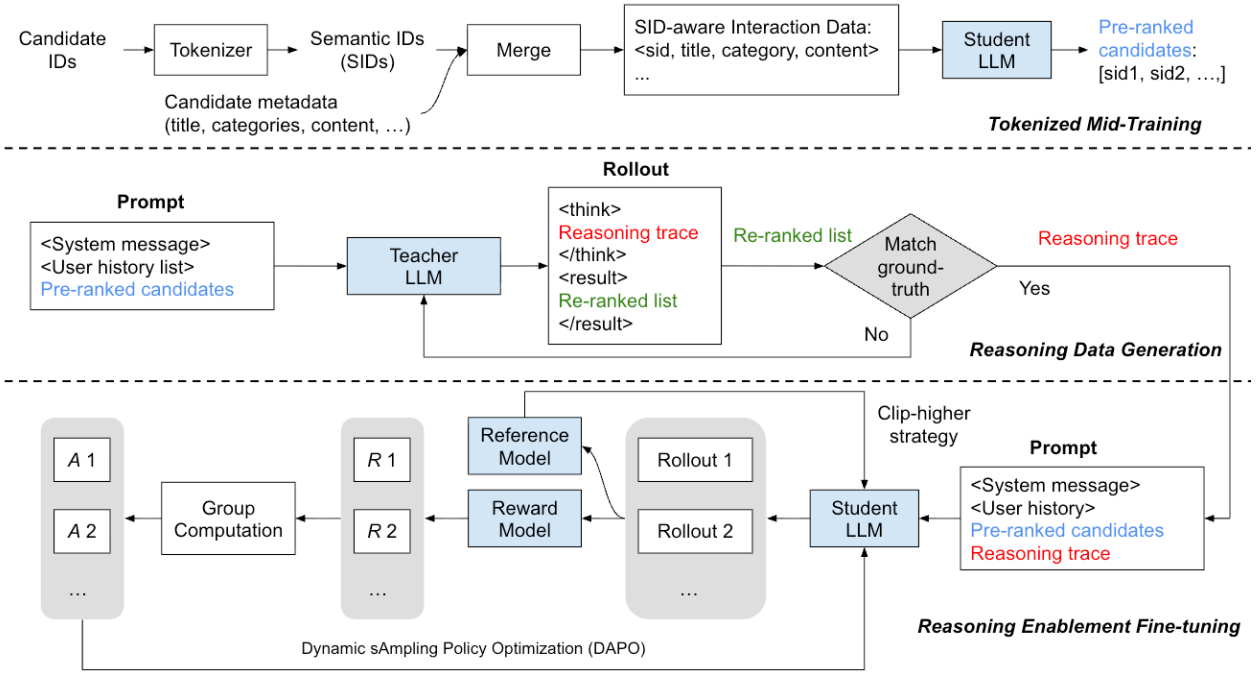

Figure 1 Overview of the 3-stage training pipeline: student LLM mid-training on tokenized semantic IDs (up), reasoning data generation with teacher LLM and rejection sampling (middle), and student LLM reasoning enablement by SFT and RL (down)

- Distinct from prior work that primarily adapts LLMs for retrieval and late-stage ranking, we investigate and establish design principles for LLMs tailored to the re-ranking stage in recommendation systems.
- We introduce a robust method for transforming non-semantic item identifiers into semantic IDs, achieving $\geq 99\%$ uniqueness. Through mid-training on a data mixture of semantic IDs and world knowledge, we

enable the LLM to recognize and reason over these enriched representations, effectively bridging item semantics with external knowledge.

- Recognizing the centrality of reasoning in re-ranking, we develop specialized prompts tailored for the re-ranking context to elicit high-quality, hierarchical reasoning traces. Rejection sampling is employed to filter out noisy traces, and the model is subsequently fine-tuned on these curated traces to impart foundational reasoning capabilities.
- To further augment the model's reasoning proficiency, we adapt the DAPO algorithm with a custom reward function specifically designed for re-ranking scenarios. This stage provides scalable, reward-driven supervision, enabling the LLM to refine its reasoning process and deliver superior re-ranking performance.
- Our comprehensive evaluation of GR2 on two real-world datasets demonstrates its superior performance: GR2 consistently surpasses the leading baseline, OneRec-Think, across both Recall@K and NDCG@K metrics. Furthermore, ablation studies validate the vital contributions of reasoning and RL, uncovering additional insights into our design.

The remainder of this paper is structured as follows. Section 2 details the first-stage tokenized mid-training process, including the encoding of engagement signals via contrastive loss and techniques for achieving high uniqueness. Section 3 elaborates on the generation of reasoning data, such as prompt engineering to foster re-ranking-oriented reasoning and the application of rejection sampling techniques to denoise. In Section 4, we present methods for enabling reasoning capabilities through advanced RL, featuring reward functions specifically tailored for re-ranking tasks. Section 6 provides a concise review of related work in LLM-based recommendation systems and LLM reasoning. Experiments and analyses are carried out in Section 5. Finally, Section 7 concludes the paper, with additional details provided in the Appendix.

2 Tokenized Mid-Training

In this section, we introduce the key components for attaining the LLM-based generative recommendation model: **Tokenization** and **Mid-Training**. To improve the cookbook utilization, we introduce our enhancement for the SID generation. To obtain the generative retriever, we stemmed from the mid-training workflow from the state-of-the-art LLM-based generative retriever that perform multi-task post-training on a LLM.

2.1 Tokenizer and Semantic ID (SID)

The technique known as semantic ID (SID) has been widely adopted in recommendation systems to mitigate the scalability issues of large embedding tables arising from massive item and user vocabularies. The seminal work TIGER (Rajput et al., 2023) is the first to apply this technique to sequential recommendation. At a high level, given the textual feature x of an item, the tokenizer maps it to a sequence of discrete integers, i.e., a compact symbolic representation, defined as

$$\boxed{\text{Tokenizer}(x) = (z_1, z_2, \ldots, z_K), \quad z_k \in 1, \ldots, C_k,} \tag{1}$$

where C_i is the i-th codebook. The core of the tokenizer is a Residual-Quantized Variational Autoencoder (RQ-VAE) (Lee et al., 2022). Since SID itself is not the primary contribution of this paper, we defer the detailed formulation to Appendix A.

2.2 Contrastive Loss

An effective way to encode the item co-engagement history is to apply a contrastive loss. For the i-th anchor item x_i the contrastive loss is presented as:

$$\mathcal{L}_{\text{ctr}}(x_i) = -\frac{1}{|\mathcal{P}(i)|} \sum_{p \in \mathcal{P}(i)} \log \frac{\exp(s(x_i, x_p))}{\exp(s(x_i, x_p)) + \sum_{n \in \mathcal{N}(i)} \exp(s(x_i, x_n))}. \tag{2}$$

where $\mathcal{P}(i)$ denote the set of indices of co-engaged (positive) samples, $\mathcal{N}(i)$ denote a set of randomly sampled non-co-engaged (negative) samples. The similarity between two items x_i and x_j is defined as a temperature

$(T > 0)$-scaled cosine similarity:

$$s(x_i, x_j) = \frac{\tilde{\mathbf{h}}_i^\top \tilde{\mathbf{h}}_j}{T}, \quad \text{where } \tilde{\mathbf{h}} = \frac{\mathbf{h}}{\|\mathbf{h}\|_2}. \tag{3}$$

2.3 Techniques for Balanced Codebook Utilization

A key challenge in training RQ-VAE (Lee et al., 2022) is *codebook collapse*, where only a small subset of codebook entries are actively used while the rest remain "dead." This leads to poor reconstruction quality and high collision rates in the generated semantic IDs. We employ five complementary techniques to ensure balanced codebook utilization.

RQ-K-means Initialization. Rather than randomly initializing codebooks, we apply the k-means++ (Arthur and Vassilvitskii, 2007) clustering on the residuals, e.g., RQ-K-Means to initialize the codebooks. This is our default setup and is applied in all of our configures.

Exponential Moving Average (EMA) Codebook Updates. Instead of updating codebook entries via gradient descent, we use exponential moving averages for updating the k-th codebook:

$$\tilde{\mathbf{e}}^{(k)} \leftarrow \texttt{Optimizer}(\mathbf{e}^{(k)}, \nabla_{\mathbf{e}^{(k)}} \mathcal{L}) \tag{4}$$

$$\mathbf{e}^{(k)} \leftarrow \gamma \mathbf{e}^{(k)} + (1-\gamma)\tilde{\mathbf{e}}^{(k)} \tag{5}$$

where $\gamma \in [0.95, 0.99]$ is the decay rate. EMA updates provide more stable learning for the codebook by avoiding direct gradient-based optimization.

Diversity Loss. To encourage uniform codebook usage, we add a diversity regularization term that penalizes uneven assignment probabilities:

$$p_k = \frac{n_k}{\sum_{j=1}^{K} n_j}, \quad \mathcal{L}_{\text{div}} = \lambda_{\text{div}} \cdot K \sum_{k=1}^{K} p_k^2 \tag{6}$$

where λ_{div} is the diversity weight; n_k is The number of input vectors in the current mini-batch that are assigned to codebook entry k, which is approximated by soft probabilities using softmax over negative distances. This loss is minimized when $p_k = 1/K$ for all k (uniform distribution), encouraging the model to utilize all codebook entries equally.

Dead Code Reset. We track the number of consecutive batches each codebook entry has been unused. When a code remains unused for more than τ batches (the reset threshold), we reinitialize it with a "hard" sample—one that has the largest reconstruction error under the current codebook. Here we omit the codebook level, i.e., the superscript, for brevity. Suppose for the k-th codebook, if $\texttt{unused\_count}[k] \geq \tau$

$$\mathbf{e}_k \leftarrow \mathbf{r}_{i^*} \tag{7}$$

$$\textit{where} \quad i^* = \arg\max_i \|\mathbf{r}_i - \mathbf{e}_{z_i}\|_2 \tag{8}$$

This mechanism revives dead codes by assigning them to poorly-fitted samples, improving overall codebook coverage.

Random Last Levels. To further improve semantic ID uniqueness while preserving semantic meaning in the early levels, we introduce random assignment for the last M levels during inference:

$$k_\ell = \begin{cases} \arg\min_k \|\mathbf{r}^{(\ell)} - \mathbf{e}_k^{(\ell)}\|^2 & \text{if } \ell \leq L - M \\ \texttt{Uniform}(1, K) & \text{if } \ell > L - M \end{cases} \tag{9}$$

The intuition is that early levels capture coarse semantic categories (e.g., product type), while later levels encode finer details. By randomizing the last M levels, we trade off some reconstruction fidelity for guaranteed uniqueness of the generated IDs.

2.4 Mid-Training through Multi-Task Training Strategy

TIGER (Rajput et al., 2023) trains an autoregressive model purely over Semantic ID sequences. In the era of large language models, leveraging world knowledge encoded in pretrained LMs has become an important direction. To this end, OneRec-Think (Liu et al., 2025) developed by Kuaishou introduces a mid-training stage (termed as 'item alignment' stage) for the pretrained LLM, a key design that enables the LLM to align the recommendation knowledge and language (Semantic IDs) with the LLM's linguistic space and world knowledge.

The key idea of the mid-training stage is to **interleave Semantic IDs (SIDs) with natural language tokens within a single sequence** and to optimize the SID embedding table through the **next-token prediction objective**. We follow the setup of item alignment from the OneRec-Think (Liu et al., 2025) and present examples of item alignment tasks in Appendix B. In addition, Google's PLUM (He et al., 2025) incorporates a similar training stage, referred to as **Continued Pre-training (CPT)**, which follows the same core idea as item alignment.

3 Reasoning Data Generation

In this section, we elucidate the design of reasoning data generation for the re-ranking task. In Sec. 3.1, we introduce a chat-format training sample structure that grounds item representation in semantic IDs and enables chain-of-thought reasoning with structured JSON output. In Sec. 3.2, we present two complementary strategies for generating high-quality reasoning traces—targeted sampling, which leverages ground-truth guidance, and rejection sampling, which ensures reasoning authenticity through iterative verification—along with five prompt design principles that elicit grounded, interpretable, and domain-aware reasoning from large language models.

3.1 Chat-formatted Template of Reasoning Training Data

To train the LLM for re-ranking with semantic ID reasoning, we construct training samples in a chat-format structure consisting of three message roles. The design follows six key principles:

- **Role-based System Prompt**: The system message establishes an expert persona (e.g., "professional recommendation expert") and specifies the re-ranking objective, activating the model's domain-specific reasoning capabilities.
- **SID-grounded Item Representation**: Both purchase history items and candidate items are tagged with semantic ID tokens (e.g., `<|sid_begin|><s_a_57>...<|sid_end|>`), enabling the model to learn reasoning patterns that naturally interleave semantic IDs with natural language.
- **Rich Item Metadata**: Each item is accompanied by its title and category hierarchy, providing multi-modal context that supports both semantic understanding and category-aware reasoning.
- **Unified Item Format**: Purchase history and candidate items share the same structural format (SID + title + categories), ensuring consistent representation learning across input contexts.
- **Chain-of-Thought Reasoning Trace**: The assistant response contains step-by-step reasoning, that explicitly cites items by their SIDs, teaching the model to produce grounded, verifiable reasoning traces.
- **Structured JSON Output**: The response containing the reasoning trace is formatted as a JSON object containing both the reasoning explanation and a ranked recommendation list, enabling direct parsing and evaluation of model outputs.

Chat-Format Training Sample Structure

System Message – Role Definition & Task Specification

{
"role": "system",
"content": "You are a **professional e-commerce recommendation expert** specializing in sequential purchase prediction. YOUR TASK: Predict which item the user is MOST LIKELY TO PURCHASE NEXT by re-ranking 10 pre-ranked candidates from a generative retrieval model."
}

▷ *Establishes expert persona and defines the re-ranking objective*

User Message – Collaborative Context with SID-grounded Items

{
"role": "user",
"content": "
// Purchase History (SID + Title + Categories)
The user has purchased the following items:
<|sid_begin|><s_a_57>...<|sid_end|>, title: "...", categories: "...";
<|sid_begin|><s_a_7>...<|sid_end|>, title: "...", categories: "...";
// Candidate Set (SID + Title + Categories)
Please re-rank the following candidates:
Candidate 1: <|sid_begin|>...<|sid_end|>, title: "...", categories: "..."
Candidate 2: <|sid_begin|>...<|sid_end|>, title: "...", categories: "..."
... (Candidates 3–10)
"
}

▷ *Provides complete context: user history + all candidates, each with semantic ID, title, and category metadata*

Assistant Message – Structured Reasoning Trace & Ranked Output

{
"role": "assistant",
"content": "
{
"explanation": "
Step 1 History Summarization: "The user has purchased <|sid|> and <|sid|>..."
Step 2 Category Pattern: "Beauty > Hair Care"
Step 3 Reasoning: "The user prefers argan oil products..."
Based on patterns, Candidate 1 best matches...",
"recommendations": ["1", "2", "3", ..., "10"]
}
"
}

▷ *Produces SID-grounded reasoning trace + ranked recommendation list in structured JSON format*

The structured reasoning trace, i.e., embedded in the Assistant Message, is a key component for enabling reasoning capability for the re-ranking task with post-training, and we will discuss the corresponding generation strategy we explored in the next section.

3.2 Reasoning Trace Generation

To generate high-quality reasoning trace, we consider two generation strategies for the reasoning trace: *targeted sampling* and *rejection sampling*, depending on whether the ground truth is contained in the prompt.

Targeted Sampling. In the targeted approach, we provide the target item $\mathbf{s}_{v_{n+1}}$ and most recent k historical items, then query a larger LLM to generate a reasoning trace τ explaining the target interaction:

$$\tau \sim P_\theta(\cdot|\mathcal{P}_{\text{targeted}}([\mathbf{s}_{v_1}, ..., \mathbf{s}_{v_k}], [\mathbf{s}_{y_1}, ..., \mathbf{s}_{y_c}], \mathbf{s}_{v_{n+1}})), \tag{10}$$

where $\mathcal{P}_{\text{targeted}}(x, y, z)$ constructs a prompt to query the rationale for why a user who interacts with item sequence x would be most interested in z from candidate list y. As the ground truth is involved, the targeted approach always yields rationales for why the target might be favored by the user; however, it may hallucinate without genuine belief in the result.

Rejection Sampling. Alternatively, in our rejection sampling strategy, we do not provide the ground truth but keep querying the LLM to predict which item $\hat{\mathbf{s}}_{y_c}$ among the pre-ranked candidates is most likely to be the user's next interest, until the prediction matches the actual target:

$$(\tau, \hat{\mathbf{s}}_{y_c}) \sim P_\theta(\cdot \mid \mathcal{P}_{\text{rejection}}([\mathbf{s}_{v_1}, \ldots, \mathbf{s}_{v_k}], [\mathbf{s}_{y_1}, \ldots, \mathbf{s}_{y_c}])) \quad \text{subject to} \quad \hat{\mathbf{s}}_{y_c} = \mathbf{s}_{v_{n+1}}, \tag{11}$$

where $\mathcal{P}_{\text{rejection}}(x, y)$ constructs a prompt to query which item in candidate list y is most likely to be the user's next interest, given history x.

To generate high quality reasoning trace for re-ranking, we leverage following principles to design the prompt for instructing LLM for reasoning:

- **Concrete System Role and Re-ranking Task definition**: To incentive the re-ranking capability of large LLMs, we elucidate the recommendation domain, the dedicated roles for the LLM to display, and the concrete definition of the re-ranking task.
- **Collaborative Context Presentation**: To provide comprehensive decision context, we present both the user's purchase history (with structured metadata including titles and categories) and the complete candidate set simultaneously, enabling the model to perform holistic comparison rather than isolated item evaluation.
- **Domain Knowledge Priming**: To leverage sequential purchase patterns inherent in e-commerce (e.g., shampoo → conditioner → styling products), we explicitly prompt the model to consider such domain-specific heuristics, enabling it to apply common-sense reasoning about product complementarity and routine-based purchasing behavior.
- **Critical Guidelines as Output Constraint**: To ensure reasoning traces are grounded and verifiable, we impose explicit constraints such as requiring the model to cite items by their SIDs. This forces the model to anchor its reasoning in specific historical items rather than generating vague or hallucinated justifications, enabling direct traceability between reasoning steps and input data.
- **Structured Multi-Step Reasoning Format**: To elicit progressive and interpretable reasoning, we provide an explicit step-by-step output format with examples. This hierarchical structure guides the model through: (1) broad pattern recognition, (2) identifying complementary product types, and (3) matching to specific candidates, mirroring human decision-making processes.

High-Level Prompt Structure

P1: System Role & Task Definition

You are an **expert at analyzing e-commerce purchase patterns** and predicting user preferences.

Given the user's purchase history and a list of candidate items, you need to predict which candidate is **MOST LIKELY** to be the user's next purchase.

P2: Collaborative Context Presentation

=== USER PURCHASE HISTORY ===
- <|sid_begin|>...<|sid_end|>; Title: [Product A]; Categories: [...]
- <|sid_begin|>...<|sid_end|>; Title: [Product B]; Categories: [...]

(Complete history with SIDs, titles, and category metadata)

=== CANDIDATE ITEMS ===
Candidate 1: Title: [...]; Categories: [...]
Candidate 2: Title: [...]; Categories: [...]

(Full candidate set for holistic comparison)

P3: Domain Knowledge Priming

=== TASK ===
Analyze the user's purchase history and predict which candidate they are most likely to purchase next. Think step-by-step about the patterns and preferences shown in their history.

Hint: Consider sequential purchase patterns (e.g., shampoo → conditioner → styling)

P4: Critical Guidelines as Output Constraints

CRITICAL GUIDELINES:
1. **CITE ITEMS BY SID**: When referring to items in purchase history, cite them directly using their SID (e.g., <|sid_begin|><s_a_99>...<|sid_end|>).
   → *Enables grounded, verifiable reasoning traces*
2. Focus on analyzing patterns in the user's purchase history
3. Think about sequential purchase patterns

P5: Structured Multi-Step Reasoning Format

=== EXAMPLE OUTPUT FORMAT ===
Step 1: "Looking at the purchase history, <|sid|> and <|sid|> are both hair care products..."
   → *Broad pattern recognition*
Step 2: "The recent purchase of <|sid|> suggests the user is looking for..."
   → *Identify complementary product types*
Step 3: "Based on this pattern, Candidate X would be the natural next purchase..."
   → *Match to specific candidate*
Prediction: Re-ranked List

4 Reasoning Enablement for Re-Ranking

4.1 Problem Setup and Prompt Interface

We study the problem of *reasoning-enabled re-ranking* over a fixed candidate set. Given a user purchase history and a pre-ranked list of candidate items produced by a retriever, the goal is to re-rank the candidates such that the ground-truth next item is promoted.

Each training or inference instance is formatted as a chat prompt consisting of three roles: `system`, `user`, and `assistant`. We follow the same role semantics as defined in Section 3. During supervised fine-tuning (SFT), the model is trained to generate the full assistant message. During reinforcement learning (RL), the policy conditions only on the system and user messages, and the assistant output is treated as the action.

4.2 Supervised Fine-tuning with Reasoning Traces

Recommender models often struggle to generate effective CoT reasoning from noisy and lengthy real-world user behavior sequences (Liu et al., 2025). The generated reasoning traces are used to supervise a base LLM

to acquire reasoning capability for re-ranking. We fine-tune the model to generate both the reasoning trace $\tau = [r_1, \ldots, r_M]$ and the ranked output $\mathbf{o} = [o_1, \ldots, o_T]$.

To preserve ranking performance while enabling reasoning, we decouple the losses for reasoning and ranking tokens. Specifically, we apply the language modeling loss *only to the assistant message*, with separate weights for reasoning and ranking segments:

$$\mathcal{L}_{\text{SFT}} = -\lambda_r \sum_{i=1}^{M} \log P(r_i \mid \mathcal{P}, r_{<i}) - \lambda_o \sum_{j=1}^{T} \log P(o_j \mid \mathcal{P}, \tau, o_{<j}), \tag{12}$$

where $\lambda_r < \lambda_o$ balances reasoning fluency and ranking accuracy. This training procedure teaches the model to generate SID-grounded, coherent reasoning traces that connect user history with candidate comparisons, while maintaining strong re-ranking behavior.

4.3 Reinforcement Learning

While SFT enables coherent reasoning, it does not directly optimize the re-ranking objective. Building upon the distilled reasoning capabilities, we apply reinforcement learning (RL) to further refine both the reasoning process and the final ranking quality. Given a prompt $\mathcal{P}(\mathcal{H}, \mathcal{D})$, the policy π_θ generates an output $o = (\tau, \mathbf{o})$. During RL, the policy conditions only on the system and user messages; the assistant output is treated as the action. This stage aims to lift the rank of target item in the pre-ranked list using a novel reward mechanism tailored for generative recommendation.

4.3.1 Ranking Reward

We define a ranking reward that measures how much the target item is promoted by re-ranking:

$$R_{\text{rank}} = \frac{r^{\mathcal{D}}_{\mathbf{s}_{v_{n+1}}} - r^{\mathbf{o}}_{\mathbf{s}_{v_{n+1}}}}{|\mathcal{D}|}, \tag{13}$$

where $r^{\mathcal{D}}_{\mathbf{s}_{v_{n+1}}}$ and $r^{\mathbf{o}}_{\mathbf{s}_{v_{n+1}}}$ denote the ranks of the target item in the pre-ranked and re-ranked lists, respectively.

4.3.2 Conditional Format Reward

To ensure parseable outputs, we introduce a format reward $R_{\text{fmt}} = \Omega(o)$, which checks whether both the reasoning trace τ and ranking output $\mathbf{o}$ can be reliably extracted. The details of the parsing function is illustrated in Appendix E. However, naively combining R_{rank} and R_{fmt} may lead to reward hacking, where the model preserves the original ranking to obtain format reward. We therefore apply the format reward conditionally:

$$R = \begin{cases} R_{\text{rank}} + \alpha R_{\text{fmt}}, & \text{if } R_{\text{rank}} > 0 \text{ or } r^{\mathcal{D}}_{\mathbf{s}_{v_{n+1}}} = 1, \\ R_{\text{rank}}, & \text{otherwise.} \end{cases} \tag{14}$$

Here α is a weight hyperparameter of the format reward.

4.3.3 Training via DAPO

We optimize the policy using Decoupled Clip and Dynamic sAmpling Policy Optimization (DAPO) algorithm (Yu et al., 2025), a recently state-of-the-art RL algorithm developed upon Grouped Policy Optimization (GRPO) (Shao et al., 2024). It can effectively resolve the entropy collapse phenomenon and rollout length bias identified in the GRPO training process. For each prompt, we sample a group of G outputs $\{o_i\}_{i=1}^{G}$ and optimize:

$$\mathcal{J}_{\text{DAPO}}(\theta) = \mathbb{E}_{(q,D)\sim\mathcal{D},\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{\sum_{i=1}^{G}|o_i|} \sum_{i=1}^{G}\sum_{t=1}^{|o_i|} \min\left(r_{i,t}(\theta)\hat{A}_{i,t}, \text{clip}(r_{i,t}(\theta), 1-\varepsilon_{\text{low}}, 1+\varepsilon_{\text{high}})\hat{A}_{i,t} \right) \right] \tag{15}$$

Table 1 Statistics of the Amazon Review datasets used for sequential recommendation.

Dataset	#Users	#Items	Avg. Seq. Len.
Beauty	22,363	12,101	8.87
Sports	35,598	18,357	8.32

s.t. $0 < |\{o_i | \text{is\_equivalent}(a, o_i)\}| < G$, where

$$r_{i,t}(\theta) = \frac{\pi_\theta(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q, o_{i,<t})}, \quad \hat{A}_{i,t} = \frac{R_i - \text{mean}(\{R_i\}_{i=1}^G)}{\text{std}(\{R_i\}_{i=1}^G)}. \tag{16}$$

π_θ and $\pi_{\theta_{\text{old}}}$ denote the current and previous policies, respectively. The pair $(q, D) \sim \mathcal{D}$ is sampled from the training distribution. The advantage estimate $\hat{A}_{i,t}$ is computed for each output sequence o_i and normalized within the group G. Adhering to the Clip-Higher strategy, DAPO decouples the lower and higher clipping range as ε_{low} and $\varepsilon_{\text{high}}$). Additionally, to avoid zero policy gradients (advantages) and improve sample efficiency, it oversamples and filters out prompts with the accuracy equal to 1 and 0.

5 Experiments

In this section, we conduct comprehensive experiments to answer the following research questions:

- **Q1**: What techniques are essential for generating semantic IDs with high uniqueness, and how do different codebook configurations affect downstream recommendation performance?
- **Q2**: Does the proposed tokenization enhancement (Section 2.2 and 2.3) and mid-training strategy improve recommendation quality compared to existing state-of-the-art LLM-based approaches?
- **Q3**: How do different reasoning trace generation strategies (targeted vs. rejection sampling) and training paradigms (SFT vs. RL) affect re-ranking performance, and is reinforcement learning necessary for translating reasoning capabilities into re-ranking improvements?

5.1 Experiment Setup

5.1.1 Datasets

We evaluate our models on two Amazon Review datasets (Beauty and Sports) derived from the large-scale Amazon product review corpus curated by McAuley et al. (2015), which contains user–item interactions with rich side information such as ratings, timestamps, and product metadata. This corpus has been widely adopted as a benchmark for recommendation research due to its scale, diversity of product domains, and realistic user behavior logs. The statistics of the datasets are presented in Table 1. All datasets are preprocessed following standard protocols (Rajput et al., 2023): users and items with fewer than 5 interactions are filtered out[1] , interactions are sorted chronologically for each user, and a leave-one-out strategy is employed to construct validation and test sets.

5.1.2 Metrics

We evaluate sequential recommendation performance using standard metrics: *Recall@K* and *Normalized Discounted Cumulative Gain at K (NDCG@K)*.

To evaluate the quality of the learned SIDs, we measure the *Collision/Uniqueness Rate*, which quantifies how often multiple items are assigned the same SID.

Detailed introduction to the metrics are presented in Appendix G.

[1] https://cseweb.ucsd.edu/~jmcauley/datasets/amazon/links.html

5.2 Tokenizer Selection

5.2.1 High SID Uniqueness

Table 2 Ablation study on techniques for improving codebook uniformity in RQ-VAE. We report unique Uniqueness rate (%) on the Amazon Beauty dataset. Configurations achieving ≥99% are **bolded**.

Diversity Loss	Dead Code Reset	EMA Update	Random Last Level	Contrastive Loss	Uniqueness Rate (%)
Baseline Configurations (EMA Enabled)					
	✓	✓			98.77
	✓	✓		✓	98.28
	✓	✓	✓		**99.98**
	✓	✓	✓	✓	**99.98**
✓	✓	✓			98.92
✓	✓	✓		✓	97.89
✓	✓	✓	✓		**99.98**
✓	✓	✓	✓	✓	**99.95**
Without Dead Code Reset (EMA Enabled)					
		✓			93.74
		✓		✓	95.92
		✓	✓		**99.67**
		✓	✓	✓	**99.84**
✓		✓			94.51
✓		✓		✓	95.92
✓		✓	✓		**99.78**
✓		✓	✓	✓	**99.86**
Without EMA Update (Codebook Collapse)					
	✓				21.39
	✓			✓	6.35
	✓		✓		66.26
	✓		✓	✓	32.72
					21.76
				✓	3.91
			✓		60.71
			✓	✓	51.43
✓	✓				21.35
✓	✓			✓	3.16
✓	✓		✓		59.17
✓	✓		✓	✓	45.62
✓					19.64
✓				✓	4.54
✓			✓		68.39
✓			✓	✓	40.85

We conduct an ablation study to analyze the impact of each technique on SID uniqueness, whose results are presented in Table 2. Our findings reveal two categories of techniques based on their effectiveness:

- **Essential Techniques:**
  - **EMA Update** is critical for preventing codebook collapse. Enabling EMA improves the average SID rate from 31.32% to 97.50% (+66.2%), making it the most impactful technique.
  - **Random Last Level** significantly boosts uniqueness by randomly assigning the final quantization level

during inference. This technique improves the average SID rate from 54.06% to 82.76% (+28.7%), and all configurations achieving ≥99% uniqueness use this technique.

- **Optional Techniques (marginal or negative effect):**
  - **Diversity Loss** (λ=0.1) has minimal impact (+0.6%), suggesting that EMA updates already provide sufficient codebook utilization.
  - **Dead Code Reset** (τ=2) shows a slight negative effect (−4.0%), indicating that resetting unused codes may disrupt learned representations.
  - **Contrastive Loss** unexpectedly hurts uniqueness (−13.2%), likely because it encourages similar items to share codes, increasing collisions.

In summary, achieving ≥99% unique SID rate requires both EMA Update and Random Last Level, while the other techniques provide marginal or even negative contributions.

5.2.2 Best-Performing SID

While achieving a high unique SID rate (≥99%) is necessary to ensure each item receives a distinct identifier, it does not guarantee that the resulting semantic IDs are *semantically meaningful* for downstream recommendation tasks. To evaluate the quality of different codebook configurations, we train a TIGER-style (Rajput et al., 2023) sequential recommendation model on the Amazon Beauty dataset. The model takes a user's purchase history—represented as a sequence of semantic IDs—and autoregressively predicts the next item's semantic ID using beam search decoding. We then convert the predicted SIDs back to raw item IDs and compute standard ranking metrics (Recall@K and NDCG@K) against the ground-truth next item. Table 3 reports results for all eight configurations that achieved ≥99% unique SID rate. We observe that **contrastive loss consistently improves recommendation performance** across all settings, with the best configuration (no diversity loss, dead code reset enabled, EMA update, random last level, and contrastive loss) achieving Recall@10 of 0.0367 and NDCG@10 of 0.0190. In the other datasets, we follow this best-performing config.

Table 3 Ablation study on codebook balancing techniques for RQ-VAE on Amazon Beauty dataset. All configurations use EMA updates and random last level assignment, which are required to achieve ≥99% unique SID rate. Metrics are evaluated using a TIGER-style sequential recommendation model trained for 50 epochs.

Diversity Loss	Dead Code Reset	EMA Update	Random Last Level	Contrastive Loss	Recall @5	Recall @10	NDCG @5	NDCG @10
	✓	✓	✓		0.0197	0.0314	0.0126	0.0164
	✓	✓	✓	✓	**0.0227**	**0.0367**	**0.0145**	**0.0190**
		✓	✓		0.0187	0.0281	0.0116	0.0146
		✓	✓	✓	0.0209	0.0343	0.0128	0.0171
✓	✓	✓	✓		0.0204	0.0314	0.0131	0.0167
✓	✓	✓	✓	✓	0.0226	0.0359	0.0138	0.0181
✓		✓	✓		0.0198	0.0308	0.0123	0.0158
✓		✓	✓	✓	0.0212	0.0336	0.0132	0.0172

5.3 Results of the Tokenized Mid-Training

In Table 4 we demonstrate the performance improvement achieved by our tokenization enhancement and the mid-training compared to the existing state-of-the-art, i.e., OneRec-Think (ORT) (Liu et al., 2025), on the Amazon Beauty dataset. The "Base" and "Base+IA" are defined the same as in the OneRec-Think (ORT) (Liu et al., 2025) paper, i.e., "Base" denotes the model tuned by the raw itemic token sequence while the "Base+IA" denotes the model enhanced with Itemic Alignment. These are two different training approach to empower the recommendation capability on LLM before the reasoning-based post-training. We observe that: (1) our tokenization enhancement alone yields consistent improvements over the ORT Base" model across all metrics, with relative gains of 6.7% on recall@5, 6.3% on recall@10, 4.5% on ndcg@5, and 4.2% on ndcg@10, demonstrating that our semantic tokenization strategy better captures item representations; (2) when combined

with mid-training, both our single-task and multi-task learning approaches substantially outperform the ORT Base+IA" baseline on recall@5, recall@10, and ndcg@5, with the most notable improvement being 18.7% on ndcg@5 for single-task learning; (3) multi-task learning, despite training on additional tasks beyond sequential preference prediction, achieves on-par or superior results, notably a 5.3% improvement on recall@10, highlighting the success of the multi-task training framework in balancing diverse objectives without sacrificing performance on the primary recommendation task.

Table 4 Performance comparison with the OneRec-Think (ORT) (Liu et al., 2025) on the Amazon Beauty dataset. Our proposed tokenization enhancement and the corresponding mid-training achieve significant improvement over the ORT's results provided by their official GitHub implementation (https://github.com/wangshy31/OneRec-Think). The result of "Base" and "Base+IA" are cited from the Table 2 of the ORT paper.

Methods	recall@5	recall@10	ndcg@5	ndcg@10
Base (ORT (Liu et al., 2025))	0.0460	0.0654	0.0314	0.0377
tokenization enhancement (Ours)	**0.0491**	**0.0695**	**0.0328**	**0.0393**
Base+IA (ORT (Liu et al., 2025))	0.0532	0.0735	0.0342	**0.0471**
Mid-Training w/ Single-task (Ours)	**0.0564**	0.0744	**0.0406**	0.0467
Mid-Training w/ Multi-task (Ours)	0.0561	**0.0774**	0.0396	0.0467

5.4 Results of Re-Ranking

Table 5 Re-ranking performance comparison with retriever on the Amazon Beauty dataset. Our post-training achieve significant improvement over pre-ranked results. "MTL" denotes mid-training with multi-task. "RL-zeroshot" means RL directly from the base model without SFT. "targeted" and "rejection" correspond to the two reasoning trace generation strategies used to SFT and serve as the reference policies in RL stage. "KP" denotes domain knowledge priming in reasoning generation.

Methods	recall@1	recall@5	recall@9	ndcg@5	ndcg@10
Pre-rank (MTL)	0.2892	0.7227	0.9534	0.5101	0.5997
SFT-targeted-KP	0.2006	0.7250	0.9545	0.4759	0.5647
SFT-targeted-noKP	0.2761	0.7272	0.9500	0.5087	0.5964
SFT-rejection-KP	0.2784	0.7091	0.9483	0.4970	0.5907
SFT-rejection-noKP	0.2682	0.7216	0.9574	0.5003	0.5904
RL-zeroshot	0.2801	0.7074	0.9574	0.4982	0.5926
RL-targeted-KP	0.2898	0.7221	0.9534	0.5101	0.5998
RL-targeted-noKP	0.2932	0.7318	0.9534	0.5172	0.6038
RL-rejection-KP	**0.2977**	**0.7460**	0.9563	**0.5234**	**0.6050**
RL-rejection-noKP	0.2915	0.7330	**0.9580**	0.5172	0.6036

The model obtained from mid-training is used as a *retriever* to produce a top-K pre-ranked candidate list for each prompt, which also serves as the baseline. The goal of the proposed GR2 framework is to improve recommendation quality by *re-ranking* these candidates through reasoning-aware post-training. Table 5 and Table 6 report the re-ranking results on the Amazon Beauty and Sports datasets, respectively, with $K = 10$.

For each dataset, the pre-rank baseline is selected based on recall@10 performance. Specifically, mid-training with multi-task learning achieves the best recall@10 on the Beauty dataset and is therefore used to generate the candidate lists for re-ranking, while the single-task mid-trained model is used for the Sports dataset. Both the retriever and the re-ranker are initialized from Qwen3-8B. We make the following observations: (1) Reasoning trace quality plays a critical role in re-ranking performance: on the Beauty dataset, rejection sampling with hierarchical item information (RL-untargeted-KP) consistently outperforms vanilla reasoning traces (RL-zeroshot) under the same RL setting, yielding relative improvements of 3.00% in recall@1, 3.30% in recall@5, and 2.60% in ndcg@5. This demonstrates that structured and informative reasoning traces provide

Table 6 Re-ranking performance comparison with retriever on the Amazon Sports dataset. "STL" denotes mid-training with single-task.

Methods	recall@1	recall@5	recall@9	ndcg@5	ndcg@10
Pre-rank (STL)	0.2422	0.7077	0.9583	0.4796	0.5742
SFT-targeted-KP	0.1790	0.7025	0.9557	0.4491	0.5454
SFT-targeted-noKP	0.2279	0.7103	**0.9596**	0.4750	0.5688
SFT-rejection-KP	0.2363	0.7090	0.9525	0.4723	0.5664
SFT-rejection-noKP	0.2233	0.6608	0.9427	0.4437	0.5532
RL-zeroshot	**0.2461**	0.7012	0.9447	0.4772	0.5732
RL-targeted-KP	0.2428	**0.7083**	0.9583	**0.4802**	**0.5745**
RL-targeted-noKP	0.2383	**0.7083**	0.9583	0.4786	0.5730
RL-rejection-KP	0.2415	0.7077	0.9583	0.4791	0.5737
RL-rejection-noKP	0.2370	0.7044	0.9583	0.4743	0.5701

a stronger learning signal for ranking refinement; (2) RL further enhances reasoning-activated models: when applied on top of reasoning-aware SFT, on the sports dataset, RL-targeted-KP improves over the pre-rank baseline by 0.24% recall@1 and 0.13% ndcg@5, indicating that policy optimization can refine the reasoning process to better align with ranking objectives; (3) Improving re-ranking via reasoning is non-trivial and requires RL: although SFT learns high-quality and coherent reasoning traces, it does not consistently translate into improved re-ranking performance, and in some cases even degrades recall@1. This highlights the mismatch between reasoning quality and ranking optimality, and underscores the necessity of RL to explicitly optimize the reasoning action space toward ranking-aware rewards.

6 Related Works

6.1 Generative LLM RecSys

Recent work has increasingly reformulated recommendation as a sequence generation problem using large language models. TIGER (Rajput et al., 2023) pioneered this direction by introducing Semantic IDs (SIDs), which discretize item representations via RQ-VAE and enable autoregressive recommendation without large embedding tables. Building on the generative formulation, OneRec (Zhou et al., 2025b) proposed a unified text-to-text framework that casts multiple recommendation tasks into a single LLM-based generation paradigm. To further enhance model capability, OneRec-Think (Liu et al., 2025) incorporated explicit reasoning traces, showing that chain-of-thought-style intermediate reasoning can improve recommendation accuracy and robustness. Complementarily, OpenOneRec (Zhou et al., 2025a) focused on reproducibility and extensibility by providing a standardized research framework for generative recommenders. PLUM (He et al., 2025) shares a similar idea to OneRec-Think in using continued pre-training to align pre-trained LLMs with recommendation domains and further introduces a task-specific fine-tuning objective for generative retrieval. Together, these works characterize the evolution of generative LLM-based recommenders, from representation tokenization to reasoning- and planning-aware generation.

In contrast, our GR2 inherits key design elements such as semantic IDs and item alignment training, while placing special emphasis on (1) tokenizers that achieve higher uniqueness in semantic ID representations, (2) high-quality reasoning trace generation via carefully designed prompts and rejection sampling, and (3) tailored DAPO-based optimization and reward functions specifically designed for the re-ranking problem.

6.2 Reasoning LLM

Chain-of-thought (CoT) prompting has emerged as a foundational technique to elicit multi-step reasoning from LLMs by generating intermediate steps before final answers (Wei et al., 2022). Subsequent work has formalized and extended reasoning structures beyond CoT (Xia et al., 2025), including strategic generation

of reasoning texts to stabilize performance (Wang et al., 2024). RL approaches such as GRPO and its variants have been shown to further enhance the faithfulness and coherence of LLM reasoning by optimizing verifiable reward signals during training (Shao et al., 2024; Yu et al., 2025; Zheng et al., 2025; Chen et al., 2025). These algorithms provide a broader context for our optimization strategy designed for reasoning-guided re-ranking. LLM-based document re-ranking in search domain is the study closest our work. For instance, ReaRank explicitly reasons before re-ranking passage lists via RL, achieving improved relevance and interpretability (Zhang et al., 2025a). Rank-R1 enhances LLM-based rerankers through RL-based reasoning optimization on document ranking benchmarks (Feng et al., 2025). Similarly, ReasonRank and MM-R5 explore reasoning-augmented reranking in multi-view and multi-modal settings (Xu et al., 2025), highlighting the promise of reasoning-aware ranking agents. R^4ec (Gu et al., 2025) employs LLM reasoning capability to iteratively reflection and refine recommendation. LLMs are used to enhance re-ranking accuracy and interpretability by applying advanced reasoning and a bootstrapping mechanism that randomizes candidate lists, which has been shown to mitigate position bias and promote fairness (Wang et al., 2025). Our GR2 differs in its focus on semantic IDs and tailored reward functions for recommendation re-ranking, leveraging structured reasoning traces and rejection sampling for higher-quality supervision.

7 Conclusion

We propose Generative Reasoning Re-ranker (GR2), a novel framework that elevates the re-ranking stage in recommendation systems by fully leveraging the reasoning capabilities of large language models. GR2 introduces a three-stage pipeline: (1) mid-training on highly unique semantic IDs to bridge item semantics and world knowledge, (2) generation and supervised fine-tuning on high-quality reasoning traces to impart foundational reasoning skills, and (3) reinforcement learning with a custom reward function tailored for re-ranking, ensuring robust and scalable supervision. Our extensive experiments on real-world datasets demonstrate that GR2 consistently surpasses strong baselines in both recall and ranking metrics, with ablation studies highlighting the importance of advanced reasoning and RL objectives. By integrating semantic representations, structured reasoning, and reward-driven optimization, GR2 sets a new benchmark for interpretable and effective re-ranking in large-scale recommendation systems, paving the way for future research in reasoning-aware recommendation models.

References

David Arthur and Sergei Vassilvitskii. k-means++: The advantages of careful seeding. In *Proceedings of the Nineteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1027–1035, 2007.

Kayhan Behdin, Yun Dai, Ata Fatahibaarzi, Aman Gupta, Qingquan Song, Shao Tang, Hejian Sang, Gregory Dexter, Sirou Zhu, Siyu Zhu, et al. Efficient ai in practice: Training and deployment of efficient llms for industry applications. 2025.

Aili Chen, Aonian Li, Bangwei Gong, Binyang Jiang, Bo Fei, Bo Yang, Boji Shan, Changqing Yu, Chao Wang, Cheng Zhu, et al. Minimax-m1: Scaling test-time compute efficiently with lightning attention. *arXiv preprint arXiv:2506.13585*, 2025.

Jiaxin Deng, Shiyao Wang, Kuo Cai, Lejian Ren, Qigen Hu, Weifeng Ding, Qiang Luo, and Guorui Zhou. Onerec: Unifying retrieve and rank with generative recommender and iterative preference alignment. *arXiv preprint arXiv:2502.18965*, 2025.

Tao Feng, Zhigang Hua, Zijie Lei, Yan Xie, Shuang Yang, Bo Long, and Jiaxuan You. Iranker: Towards ranking foundation model. *arXiv preprint arXiv:2506.21638*, 2025.

Jingtong Gao, Xiangyu Zhao, Muyang Li, Minghao Zhao, Runze Wu, Ruocheng Guo, Yiding Liu, and Dawei Yin. Smlp4rec: An efficient all-mlp architecture for sequential recommendations. *ACM Transactions on Information Systems*, 42(3):1–23, 2024.

Jingtong Gao, Bo Chen, Xiangyu Zhao, Weiwen Liu, Xiangyang Li, Yichao Wang, Wanyu Wang, Huifeng Guo, and Ruiming Tang. Llm4rerank: Llm-based auto-reranking framework for recommendations. In *Proceedings of the ACM on Web Conference 2025*, pages 228–239, 2025.

Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In *Proceedings of the 16th ACM Conference on Recommender Systems*, pages 299–315, 2022.

Hao Gu, Rui Zhong, Yu Xia, Wei Yang, Chi Lu, Peng Jiang, and Kun Gai. R 4ec: A reasoning, reflection, and refinement framework for recommendation systems. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems*, pages 411–421, 2025.

Ruining He, Lukasz Heldt, Lichan Hong, Raghunandan Keshavan, Shifan Mao, Nikhil Mehta, Zhengyang Su, Alicia Tsai, Yueqi Wang, Shao-Chuan Wang, et al. Plum: Adapting pre-trained language models for industrial-scale generative recommendations. *arXiv preprint arXiv:2510.07784*, 2025.

Doyup Lee, Chiheon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. Autoregressive image generation using residual quantization. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11523–11532, 2022.

Mingfu Liang, Xi Liu, Rong Jin, Boyang Liu, Qiuling Suo, Qinghai Zhou, Song Zhou, Laming Chen, Hua Zheng, Zhiyuan Li, et al. External large foundation model: How to efficiently serve trillions of parameters for online ads recommendation. In *Companion Proceedings of the ACM on Web Conference 2025*, pages 344–353, 2025.

Weiwen Liu, Yunjia Xi, Jiarui Qin, Fei Sun, Bo Chen, Weinan Zhang, Rui Zhang, and Ruiming Tang. Neural re-ranking in multi-stage recommender systems: A review.

Zhanyu Liu, Shiyao Wang, Xingmei Wang, Rongzhou Zhang, Jiaxin Deng, Honghui Bao, Jinghao Zhang, Wuchao Li, Pengfei Zheng, Xiangyu Wu, Yifei Hu, Qigen Hu, Xinchen Luo, Lejian Ren, Zixing Zhang, Qianqian Wang, Kuo Cai, Yunfan Wu, Hongtao Cheng, Zexuan Cheng, Lu Ren, Huanjie Wang, Yi Su, Ruiming Tang, Kun Gai, and Guorui Zhou. Onerec-think: In-text reasoning for generative recommendation. *CoRR*, abs/2510.11639, 2025. doi: 10.48550/ARXIV.2510.11639. https://doi.org/10.48550/arXiv.2510.11639.

Liang Luo, Yuxin Chen, Zhengyu Zhang, Mengyue Hang, Andrew Gu, Buyun Zhang, Boyang Liu, Chen Chen, Chengze Fan, Dong Liang, et al. Meta lattice: Model space redesign for cost-effective industry-scale ads recommendations. *arXiv preprint arXiv:2512.09200*, 2025.

Julian McAuley, Christopher Targett, Qinfeng Shi, and Anton Van Den Hengel. Image-based recommendations on styles and substitutes. In *Proceedings of the 38th international ACM SIGIR conference on research and development in information retrieval*, pages 43–52, 2015.

Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Tran, Jonah Samost, et al. Recommender systems with generative retrieval. *Advances in Neural Information Processing Systems*, 36:10299–10315, 2023.

Sudarshan Srinivasa Ramanujam, Antonio Alonso, Saurabh Kataria, Siddharth Dangi, Akhilesh Gupta, Birjodh Singh Tiwana, Manas Somaiya, Luke Simon, David Byrne, Sojeong Ha, et al. Large scale retrieval for the linkedin feed using causal language models. *arXiv preprint arXiv:2510.14223*, 2025.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Qwen Team. Qwen3 technical report, 2025. https://arxiv.org/abs/2505.09388.

Ruoxi Wang, Rakesh Shivanna, Derek Cheng, Sagar Jain, Dong Lin, Lichan Hong, and Ed Chi. Dcn v2: Improved deep & cross network and practical lessons for web-scale learning to rank systems. In *Proceedings of the web conference 2021*, pages 1785–1797, 2021.

Yaqi Wang, Haojia Sun, and Shuting Zhang. Llm as explainable re-ranker for recommendation system. *arXiv preprint arXiv:2512.03439*, 2025.

Yu Wang, Shiwan Zhao, Zhihu Wang, Heyuan Huang, Ming Fan, Yubo Zhang, Zhixing Wang, Haijun Wang, and Ting Liu. Strategic chain-of-thought: Guiding accurate reasoning in llms through strategy elicitation. *arXiv preprint arXiv:2409.03271*, 2024.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, et al. A survey on large language models for recommendation. *World Wide Web*, 27(5):60, 2024.

Yu Xia, Rui Wang, Xu Liu, Mingyan Li, Tong Yu, Xiang Chen, Julian McAuley, and Shuai Li. Beyond chain-of-thought: A survey of chain-of-X paradigms for LLMs. In Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert, editors, *Proceedings of the 31st International Conference on Computational Linguistics*, pages 10795–10809, Abu Dhabi, UAE, January 2025. Association for Computational Linguistics. https://aclanthology.org/2025.coling-main.719/.

Mingjun Xu, Jinhan Dong, Jue Hou, Zehui Wang, Sihang Li, Zhifeng Gao, Renxin Zhong, and Hengxing Cai. Mm-r5: Multimodal reasoning-enhanced reranker via reinforcement learning for document retrieval. *arXiv preprint arXiv:2506.12364*, 2025.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

Buyun Zhang, Liang Luo, Xi Liu, Jay Li, Zeliang Chen, Weilin Zhang, Xiaohan Wei, Yuchen Hao, Michael Tsang, Wenjun Wang, et al. Dhen: A deep and hierarchical ensemble network for large-scale click-through rate prediction. *arXiv preprint arXiv:2203.11014*, 2022.

Buyun Zhang, Liang Luo, Yuxin Chen, Jade Nie, Xi Liu, Shen Li, Yanli Zhao, Yuchen Hao, Yantao Yao, Ellie Dingqiao Wen, et al. Wukong: towards a scaling law for large-scale recommendation. In *Proceedings of the 41st International Conference on Machine Learning*, pages 59421–59434, 2024.

Le Zhang, Bo Wang, Xipeng Qiu, Siva Reddy, and Aishwarya Agrawal. REARANK: Reasoning re-ranking agent via reinforcement learning. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 2458–2471, Suzhou, China, November 2025a. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.125. https://aclanthology.org/2025.emnlp-main.125/.

Yihua Zhang, Xi Liu, Xihuan Zeng, Mingfu Liang, Jiyan Yang, Rong Jin, Wen-Yen Chen, Yiping Han, Hao Ma, Bo Long, et al. Reasonrec: A reasoning-augmented multimodal agent for unified recommendation. In *ICML 2025 Workshop on Programmatic Representations for Agent Learning*, 2025b.

Zihuai Zhao, Wenqi Fan, Jiatong Li, Yunqing Liu, Xiaowei Mei, Yiqi Wang, Zhen Wen, Fei Wang, Xiangyu Zhao, Jiliang Tang, et al. Recommender systems in the era of large language models (llms). *IEEE Transactions on Knowledge and Data Engineering*, 36(11):6889–6907, 2024.

Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.

Guorui Zhou, Na Mou, Ying Fan, Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. Deep interest evolution network for click-through rate prediction. In *Proceedings of the AAAI conference on artificial intelligence*, volume 33, pages 5941–5948, 2019.

Guorui Zhou, Honghui Bao, Jiaming Huang, Jiaxin Deng, Jinghao Zhang, Junda She, Kuo Cai, Lejian Ren, Lu Ren, Qiang Luo, et al. Openonerec technical report. *arXiv preprint arXiv:2512.24762*, 2025a.

Guorui Zhou, Hengrui Hu, Hongtao Cheng, Huanjie Wang, Jiaxin Deng, Jinghao Zhang, Kuo Cai, Lejian Ren, Lu Ren, Liao Yu, et al. Onerec-v2 technical report. *arXiv preprint arXiv:2508.20900*, 2025b.

Huixue Zhou, Hengrui Gu, Zaifu Zhan, Xi Liu, Kaixiong Zhou, Yongkang Xiao, Mingfu Liang, Srinivas Prasad Govindan, Piyush Chawla, Jiyan Yang, et al. The efficiency vs. accuracy trade-off: Optimizing rag-enhanced llm recommender systems using multi-head early exit. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 26443–26458, 2025c.

Appendix

A Semantic ID (SID)

Given the textual feature x of an item, a textual encoder is used to generate an embedding vector:

$$\mathbf{h} = f_{\text{enc}}(x), \quad \mathbf{h} \in \mathbb{R}^d. \tag{17}$$

The residual vector is initialized as

$$\mathbf{r}_1 = \mathbf{h}, \tag{18}$$

which is then decomposed into a set of vectors $\{\mathbf{q}_i\}$ based on the nearest neighbor search

$$z_k = \arg\min_{j \in \{1,\dots,C_k\}} \left\| \mathbf{r}_k - \mathbf{e}_j^{(k)} \right\|_2^2, \tag{19}$$

$$\mathbf{q}_k = \mathbf{e}_{z_k}^{(k)}, \tag{20}$$

$$\mathbf{r}_{k+1} = \mathbf{r}_k - \mathbf{q}_k, \qquad k = 1, \dots, K. \tag{21}$$

where $\mathbf{e}_j^{(k)}$ denotes the j-th code in the k-th codebook. The above process can be viewed as a tokenizer which generates the semantic IDs (SIDs) for the item with text x:

$$\boxed{\text{Tokenizer}(x) = (z_1, z_2, \dots, z_K), \quad z_k \in \{1, \dots, C_k\}.} \tag{22}$$

the reconstructed latent representation is

$$\hat{\mathbf{h}} = \sum_{k=1}^{K} \mathbf{q}_k. \tag{23}$$

The loss function for a single sample x is

$$\mathcal{L}(x) = \mathcal{L}_{\text{rec}} + \beta \mathcal{L}_{\text{cb}} + \gamma \mathcal{L}_{\text{com}}, \tag{24}$$

$$\mathcal{L}_{\text{rec}} = \left\| \mathbf{h} - \hat{\mathbf{h}} \right\|_2^2, \quad \mathcal{L}_{\text{cb}} = \left\| \text{sg}[\mathbf{h}] - \hat{\mathbf{h}} \right\|_2^2, \quad \mathcal{L}_{\text{com}} = \left\| \mathbf{h} - \text{sg}[\hat{\mathbf{h}}] \right\|_2^2, \tag{25}$$

where sg$[\cdot]$ is stop-gradient.

Based on SIDs, TIGER (Rajput et al., 2023) reformulates recommendation as an autoregressive generation problem over discrete item representations. Given a user's interaction history $\mathcal{H}$, a Transformer model sequentially predicts the next item's Semantic ID $(z_1, \dots, z_K)$:

$$p\big((z_1, \dots, z_K) \mid \mathcal{H}\big) = \prod_{k=1}^{K} p\big(z_k \mid \mathcal{H}, z_{<k}\big). \tag{26}$$

the training algorithm for the RQ-VAE is detailed in Algorithm 1.

At inference time, the generated Semantic ID is mapped back to candidate items via the tokenizer's inverse mapping, enabling direct item retrieval.

B Examples of Item Alignment Tasks

Here we present representative examples of **Item Alignment** tasks. OneRec-Think (Liu et al., 2025) introduces four such tasks: **(1) Interleaved User Persona Grounding, (2) Sequential Preference Modeling, (3) Itemic Dense Captioning, and (4) General Language Modeling**. An example of Itemic Dense Captioning is provided in Section 2.4. Since the General Language Modeling task is trained on general-purpose text corpora, we omit its examples for brevity.

The following prompt illustrates one of the item alignment training tasks, namely **Itemic Dense Captioning**.

Algorithm 1 Vanilla RQ-VAE Training

Input: Training set $\mathcal{X}$, codebooks $\{\mathbf{e}_j^{(k)}\}$, encoder f_{enc}, hyperparameters β, γ

Output: Updated codebooks $\{\mathbf{e}_j^{(k)}\}$

```
for each sample x ∈ X do
    Compute embedding h and initialize residual r_1 using Eqs. (17) and (18)
    for k = 1 to K do
        Generate k-th level SID z_k using Eqs (19), (20), and (21)
    end for
    Update codebooks {e_j^(k)} to minimize the objective in Eq. (24)
end for
```

Prompt Example for Itemic Dense Captioning

<|im_start|>system
You are an expert content analyst. Given an itemic token, generate a concise and accurate textual description of its content.
<|im_end|>

<|im_start|>user
Provide a description for the itemic token:
<|sid_begin|><s_a_97><s_b_168><s_c_137><s_d_135><|sid_end|>
<|im_end|>

<|im_start|>assistant
Title: Stella McCartney Stella
Description: STELLA For Women By STELLA MCCARTNEY 1.7 oz EDP Spray
Categories: Beauty > Fragrance > Women's > Eau de Parfum
<|im_end|>

We next present examples of the Interleaved User Persona Grounding and Sequential Preference Modeling tasks.

Prompt Example for Interleaved User Persona Grounding

User ID: A6F8KH0J1AVYA

 # User Profile Narrative
The average rating given by this user is 4.0 out of 5.

 # Review History
The user recently commented on the following items:

Item 1
Item SID: <|sid_begin|><s_a_60><s_b_175><s_c_86><s_d_158><|sid_end|>
Review Title: Did work for me
Review Content: I order this cream along with their soap. It actually worked for me but after I finished the tube, I ordered different brand just to get quicker results (BAD IDEA). I am definitely ordering 3 more tubes so that my underarm pigment gets treated completely.

Item 2
Item SID: <|sid_begin|><s_a_229><s_b_165><s_c_210><s_d_115><|sid_end|>
Review Title: average quality
Review Content: This oil has different consistency compare to Josie Maran argan oil. it doesn't worth the money, there is a lot of room for quality improvement.

Item 3
Item SID: <|sid_begin|><s_a_23><s_b_71><s_c_33><s_d_5><|sid_end|>
Review Title: great product
Review Content: it works really well, easy way to get perfect bun. I haven't used the small clip but larger one is really good and long lasting.

Prompt Example for Sequential Preference Modeling

User ID: A1TLDR1V4O48PK

 # Purchase History
The user has purchased the following items:

Item 1
Item SID: <|sid_begin|><s_a_52><s_b_72><s_c_153><s_d_241><|sid_end|>
Title: 120 Color Eyeshadow Palette 3rd Edition
Categories: Beauty > Makeup > Eyes > Eye Shadow

Item 2
Item SID: <|sid_begin|><s_a_221><s_b_217><s_c_124><s_d_107><|sid_end|>
Title: Ion Color Brilliance Brights Semi-Permanent Hair Color Purple
Categories: Beauty > Hair Care > Hair Color > Chemical Hair Dyes

Item 3
Item SID: <|sid_begin|><s_a_60><s_b_175><s_c_86><s_d_158><|sid_end|>
Title: Xtreme Brite Brightening Gel 1oz.
Categories: Beauty > Hair Care > Styling Products > Creams, Gels & Lotions

C Reasoning Trace Generation with Targeted Sampling

Here we present two variant of reasoning trace generation with targeted sampling.

C.1 Reasoning Trace Generation with the Context of Category Hierarchy

Prompt Example for Target Sampling with the context of Category Hierarchy

 # System Role
You are an expert at analyzing e-commerce purchase patterns and predicting user preferences. Given the user's purchase history (with SID identifiers) and a list of candidate items, reason step-by-step about which candidate is most likely to be the user's next purchase.

 # Available Category Hierarchy
Categories are structured in a hierarchy. Format: 'Level0 > Level1 > Level2 > ...'

Level 0 (Root): Beauty
Level 1 (Main): Bath & Body, Fragrance, Hair Care, Makeup, Skin Care, ...
Level 2 (Sub): Conditioners, Shampoos, Styling Products, Styling Tools, ...
Level 3 (Product Type): Creams, Hair Dryers, Irons, Mousses & Foams, ...
Level 4 (Specific): Curling Irons, Flattening Irons, ...

 # User Purchase History
The user recently purchased the following items:

Item 1
Item SID: <|sid_begin|><s_a_173><s_b_97><s_c_226><s_d_18><|sid_end|>
Title: L'Oreal Paris EverSleek Sulfate-Free Smoothing System Intense Smoothing Conditioner
Categories: Beauty > Hair Care > Conditioners

Item 2
Item SID: <|sid_begin|><s_a_155><s_b_232><s_c_47><s_d_47><|sid_end|>
Title: Dove Damage Therapy Intensive Repair Daily Super Conditioner
Categories: Beauty > Hair Care > Conditioners

Item 3
Item SID: <|sid_begin|><s_a_173><s_b_79><s_c_173><s_d_249><|sid_end|>
Title: L'Oreal Paris EverStrong Sulfate-Free Fortify System Overnight Hair Repair Treatment
Categories: Beauty > Hair Care > Hair & Scalp Treatments

Item 4
Item SID: <|sid_begin|><s_a_6><s_b_13><s_c_249><s_d_8><|sid_end|>
Title: Aussie Hair Insurance Leave-In Conditioner
Categories: Beauty > Hair Care > Conditioners

Item 5
Item SID: <|sid_begin|><s_a_27><s_b_159><s_c_86><s_d_25><|sid_end|>
Title: L'Oreal Paris EverSleek Humidity Defying Leave-In Creme
Categories: Beauty > Hair Care > Styling Products > Hair Styling Serums

... (Items 6-9 follow same format) ...

 # Candidate Items

Candidate 1: Herbal Essences Hello Hydration Conditioner
Categories: Beauty > Hair Care > Conditioners

Candidate 2: Herbal Essences Tousle Me Softly Conditioner
Categories: Beauty > Hair Care > Conditioners

Candidate 3: Remington Salon Collection Ceramic Hair Straightener
Categories: Beauty > Hair Care > Styling Tools > Irons > Flattening Irons

Candidate 4: Head & Shoulders Clinical Strength Dandruff Shampoo
Categories: Beauty > Hair Care > Shampoos

... (Candidates 5-10 follow same format) ...

 # Task
The correct answer is Candidate 3 (Remington Ceramic Hair Straightener).
Generate a step-by-step reasoning trace explaining why this candidate is the best match.

Critical Guidelines:
1. **Cite items by SID**: When referring to purchase history, use their SID directly
2. Focus on analyzing patterns in the user's purchase history
3. Do NOT use phrases like "target item" or "the target"
4. Each step: Reasoning line (with SID citations) + Category line

 # Example Output Format

Step 1 Reasoning: "Looking at the purchase history, <|sid_begin|>...<|sid_end|> and <|sid_begin|>...<|sid_end|> are both in the hair care category..."
Step 1 Category: "Beauty > Hair Care"

Step 2 Reasoning: "The recent purchases of <|sid_begin|>...<|sid_end|> (a deep conditioner) suggest..."
Step 2 Category: "Beauty > Hair Care > Conditioners"

Step 3 Reasoning: "Based on their pattern of purchasing complementary hair care products..."
Step 3 Category: "Beauty > Hair Care > Styling Tools > Irons > Flattening Irons"

 # Your Response

Generate your response following the EXACT format above with all required lines:
- Step 1 Reasoning + Step 1 Category (broad)
- Step 2 Reasoning + Step 2 Category (specific)
- Step 3 Reasoning + Step 3 Category (final: Flattening Irons)

Generated Reasoning Trace (Model Output)

{
"explanation": "

Step 1 Reasoning: "The user has a history of purchasing hair conditioners and treatments, including <|sid_begin|><s_a_173><s_b_97><s_c_226><s_d_18><|sid_end|> (a smoothing conditioner), <|sid_begin|><s_a_173><s_b_79><s_c_173><s_d_249><|sid_end|> (an overnight repair treatment), and <|sid_begin|><s_a_49><s_b_13><s_c_160><s_d_43><|sid_end|> (a smoothing balm). This indicates a focus on hair health and managing issues like frizz and breakage."

Step 1 Category: "Beauty > Hair Care"

Step 2 Reasoning: "The repeated purchases of conditioners and treatments, such as <|sid_begin|><s_a_6><s_b_13><s_c_249><s_d_8><|sid_end|> (a leave-in conditioner) and <|sid_begin|><s_a_148><s_b_12><s_c_189><s_d_62><|sid_end|> (a curl-enhancing mousse), suggest the user is invested in maintaining smooth, manageable hair. This progression from basic conditioning to styling products indicates a need for tools to maintain the desired hair texture and style."

Based on the user's purchase history and category patterns, Candidate 3 best matches the predicted category and complements their recent purchases.",

"recommendations": ["3", "1", "2", "4", "5", "6", "7", "8", "9", "10"]
}

Color Legend: Section Headers | Semantic IDs (SID) | Predictions/Answers | Guidelines | Placeholders/Think Tags

C.2 Reasoning Trace Generation without the Context of Category Hierarchy

Prompt Example for Target Sampling without the context of Category Hierarchy

 # System Role

You are an expert at analyzing e-commerce purchase patterns and predicting user preferences. Given the user's purchase history (with SID identifiers) and a list of candidate items, reason step-by-step about which candidate is most likely to be the user's next purchase.

 # User Purchase History

Item 1
Item SID: <|sid_begin|><s_a_173><s_b_97><s_c_226><s_d_18><|sid_end|>
Title: L'Oreal Paris EverSleek Sulfate-Free Smoothing System Intense Smoothing Conditioner
Categories: Beauty > Hair Care > Conditioners

Item 2
Item SID: <|sid_begin|><s_a_155><s_b_232><s_c_47><s_d_47><|sid_end|>
Title: Dove Damage Therapy Intensive Repair Daily Super Conditioner
Categories: Beauty > Hair Care > Conditioners

Item 3
Item SID: <|sid_begin|><s_a_173><s_b_79><s_c_173><s_d_249><|sid_end|>
Title: L'Oreal Paris EverStrong Sulfate-Free Fortify System Overnight Hair Repair Treatment
Categories: Beauty > Hair Care > Hair & Scalp Treatments

Item 4

Item SID: <|sid_begin|><s_a_6><s_b_13><s_c_249><s_d_8><|sid_end|>
Title: Aussie Hair Insurance Leave-In Conditioner
Categories: Beauty > Hair Care > Conditioners

Item 5
Item SID: <|sid_begin|><s_a_27><s_b_159><s_c_86><s_d_25><|sid_end|>
Title: L'Oreal Paris EverSleek Humidity Defying Leave-In Creme
Categories: Beauty > Hair Care > Styling Products > Hair Styling Serums

... (Items 6–9 follow same format) ...

 # Candidate Items

Candidate 1: Herbal Essences Hello Hydration Conditioner
Categories: Beauty > Hair Care > Conditioners

Candidate 2: Herbal Essences Tousle Me Softly Conditioner
Categories: Beauty > Hair Care > Conditioners

Candidate 3: Remington Salon Collection Ceramic Hair Straightener
Categories: Beauty > Hair Care > Styling Tools > Irons > Flattening Irons

Candidate 4: Head & Shoulders Clinical Strength Dandruff Shampoo
Categories: Beauty > Hair Care > Shampoos

... (Candidates 5–10 follow same format) ...

 # Task

The correct answer is Candidate 3 (Remington Ceramic Hair Straightener).
Generate a step-by-step reasoning trace (3 steps) explaining why this candidate is the best match.

Critical Guidelines:

1. **Cite items by SID**: When referring to purchase history, use their SID directly
2. Focus on analyzing patterns in the user's purchase history
3. Do NOT use phrases like "target item" or "the target"
4. Think about sequential purchase patterns (e.g., shampoo -> conditioner, phone -> case)

 # Example Output Format

Step 1: "Looking at the user's purchase history, <|sid_begin|>...<|sid_end|> and <|sid_begin|>...<|sid_end|> are both hair care products in the conditioning category..."

Step 2: "The recent purchase of <|sid_begin|>...<|sid_end|> (a deep conditioning treatment) suggests the user is looking for products that provide intensive hair care..."

Step 3: "Based on this pattern, a hydrating hair mask would complement their existing routine..."

 # Your Response

Format your response as:
Step 1: "<analyze the user's purchase patterns, citing specific items by their SID>"
Step 2: "<identify what type of product would complement their recent purchases, citing items by SID>"
Step 3: "<describe the specific product characteristics that make this candidate the best match>"

Generated Reasoning Trace (Model Output)

{
"explanation": "

Step 1: "Looking at the user's purchase history, <|sid_begin|><s_a_173><s_b_97><s_c_226><s_d_18><|sid_end|>, <|sid_begin|><s_a_173><s_b_79><s_c_173><s_d_249><|sid_end|>, and <|sid_begin|><s_a_27><s_b_159><s_c_86><s_d_25><|sid_end|> are all smoothing and frizz control products, indicating a strong interest in managing frizz and achieving smooth hair."

Step 2: "The user has consistently purchased hair conditioners and treatments, as seen with <|sid_begin|><s_a_155><s_b_232><s_c_47><s_d_47><|sid_end|>,

<|sid_begin|><s_a_6><s_b_13><s_c_249><s_d_8><|sid_end|>, <|sid_begin|><s_a_6><s_b_134><s_c_71><s_d_245><|sid_end|>, and <|sid_begin|><s_a_6><s_b_68><s_c_105><s_d_181><|sid_end|>, suggesting they prioritize conditioning and deep hair care."

Step 3: "Candidate 3 is a ceramic hair straightener with wide plates, which aligns with the user's focus on smoothing and frizz control. It complements their existing smoothing conditioners and treatments by providing a styling tool that enhances smoothness, making it the best match for their hair care routine."

Based on the user's purchase history and category patterns, Candidate 3 best matches the predicted category and complements their recent purchases.",

"recommendations": ["3", "1", "2", "4", "5", "6", "7", "8", "9", "10"]
}

Color Legend: Section Headers / Steps Semantic IDs (SID) Predictions / Correct Answer Guidelines Placeholders

D Reasoning Trace Generation with Rejection Sampling

D.1 Reasoning Trace Generation with the Context of Category Hierarchy

Prompt Example for Rejection Sampling with the context of Category Hierarchy

 # System Role

You are an expert at analyzing e-commerce purchase patterns and predicting user preferences.

Given the user's purchase history (with SID identifiers) and a list of candidate items, you need to predict which candidate is MOST LIKELY to be the user's next purchase.

 # Available Category Hierarchy

Categories are structured in a hierarchy. Format: 'Level0 > Level1 > Level2 > ...'

Level 0 (Root): Beauty
Level 1 (Main): Bath & Body, Fragrance, Hair Care, Makeup, Skin Care, Tools & Accessories
Level 2 (Sub): Conditioners, Shampoos, Styling Products, Hair & Scalp Treatments, ...
Level 3 (Product Type): Creams, Gels & Lotions, Hair Sprays, Oils & Serums, ...
Level 4 (Specific): Curling Irons, Flattening Irons, ...
Level 5 (Variant): Retinol, Glycolic Acid, ...

 # User Purchase History

The user recently purchased the following items:

Item 1
Item SID: <|sid_begin|><s_a_57><s_b_7><s_c_213><s_d_26><|sid_end|>
Title: One'n Only Argan Oil Leave-In Treatment
Categories: Beauty > Hair Care > Hair & Scalp Treatments

Item 2
Item SID: <|sid_begin|><s_a_7><s_b_112><s_c_18><s_d_204><|sid_end|>
Title: Hair One Cleanser and Conditioner with Argan Oil for Curly Hair 12 oz
Categories: Beauty > Hair Care > Shampoos

 # Candidate Items

Candidate 1: One 'n Only Argan Oil Styling Cream, 10 fl. oz.
Categories: Beauty > Hair Care > Styling Products > Creams, Gels & Lotions

Candidate 2: One 'n Only Argan Oil Spray Treatment 6 fl. oz
Categories: Beauty > Hair Care > Hair & Scalp Treatments

Candidate 3: Deva Devacurl One Condition Conditioner, 12 Ounce

Categories: Beauty > Hair Care > Conditioners

Candidate 4: Giovanni Hair Care - Direct Leave-In Conditioner, 8.5 fl oz
Categories: Beauty > Hair Care > Hair Relaxers > Conditioners

Candidate 5: Paul Mitchell The Conditioner, Leave-in Moisturizer, 10.14-ounce
Categories: Beauty > Hair Care > Conditioners

Candidate 6: Kinky-Curly Knot Today Leave In Conditioner/Detangler - 8 oz
Categories: Beauty > Hair Care > Conditioners

Candidate 7: Hair One Cleanser and Conditioner with Olive Oil for Dry Hair 12 oz
Categories: Beauty > Hair Care > Hair Loss Products > Conditioners

Candidate 8: It's A 10 Miracle Leave In Product, 4-Ounces
Categories: Beauty > Hair Care > Conditioners

Candidate 9: It's A 10 Miracle Moisture Shampoo, 10-Ounce Bottle
Categories: Beauty > Hair Care > Shampoos

Candidate 10: Dabur Vatika Enriched Coconut Hair Oil 150ml (Pack of 2)
Categories: Beauty > Hair Care > Hair & Scalp Treatments

 # Task

Analyze the user's purchase history and predict which candidate they are most likely to purchase next.

Critical Guidelines:

1. **Cite items by SID**: When referring to items in the purchase history, cite them directly using their SID (e.g., <|sid_begin|><s_a_99><s_b_19><s_c_220><s_d_204><|sid_end|>). This allows the model to learn to reason with semantic IDs naturally interleaved with language.
2. Focus on analyzing patterns in the user's purchase history
3. Each step should have both a Reasoning line (free-form with SID citations) AND a Category line

 # Example Output Format

Step 1 Reasoning: "Looking at the purchase history, <|sid_begin|>...<|sid_end|> and <|sid_begin|>...<|sid_end|> are both in the hair care category, indicating the user focuses on personal care."
Step 1 Category: "Beauty > Hair Care"

Step 2 Reasoning: "The recent purchases of <|sid_begin|>...<|sid_end|> (a deep conditioner) and <|sid_begin|>...<|sid_end|> (a leave-in treatment) suggest the user is building a comprehensive hair care routine focused on moisture and repair."
Step 2 Category: "Beauty > Hair Care > Conditioners"

Step 3 Reasoning: "Based on their pattern, Candidate 3 (a hydrating hair mask) would naturally complement their existing routine."
Step 3 Category: "Beauty > Hair Care > Conditioners"
Prediction: Candidate 3

 # Your Response

Generate your response following the EXACT format above. You MUST include:
– Step 1 Reasoning (free-form analysis citing items by SID) + Step 1 Category (broad)
– Step 2 Reasoning (narrowing down, citing specific items by SID) + Step 2 Category (more specific)
– Step 3 Reasoning (final prediction explanation) + Step 3 Category (final specific)
– Prediction: Candidate <number>

Reasoning:

Generated Reasoning Trace (Model Output)

{
"explanation": "

Step 1 Reasoning: "The user has purchased <|sid_begin|><s_a_57><s_b_7><s_c_213><s_d_26><|sid_end|> (a leave-in treatment) and

<|sid_begin|><s_a_7><s_b_112><s_c_18><s_d_204><|sid_end|> (a cleanser and conditioner), both of which are in the hair care category, indicating a focus on hair care products."

Step 1 Category: "Beauty > Hair Care"

Step 2 Reasoning: "The user has already purchased a cleanser and conditioner with argan oil, suggesting a preference for argan oil-based products. The leave-in treatment is also in the hair and scalp treatments category, indicating a focus on treatments. Candidate 1 and Candidate 2 are in the same category as the leave-in treatment, while Candidates 3–10 are in the conditioners or other categories. Candidate 2 is in the hair and scalp treatments category, similar to the user's previous purchase of <|sid_begin|><s_a_57><s_b_7><s_c_213><s_d_26><|sid_end|>."

Based on the user's purchase history and category patterns, Candidate 1 best matches the predicted category and complements their recent purchases.",

"recommendations": ["1", "2", "3", "4", "5", "6", "7", "8", "9", "10"]
}

Color Legend: Section Headers | Semantic IDs (SID) | Predictions/Answers | Guidelines | Placeholders/Think Tags

D.2 Reasoning Trace Generation without the Context of Category Hierarchy

Prompt Example for E-Commerce Purchase Prediction (Simple Format)

 # System Role

You are an expert at analyzing e-commerce purchase patterns and predicting user preferences.

Given the user's purchase history (with SID identifiers) and a list of candidate items, you need to predict which candidate is MOST LIKELY to be the user's next purchase.

 # User Purchase History

The user recently purchased the following items:

Item 1
Item SID: <|sid_begin|><s_a_57><s_b_7><s_c_213><s_d_26><|sid_end|>
Title: One'n Only Argan Oil Leave-In Treatment
Categories: Beauty > Hair Care > Hair & Scalp Treatments

Item 2
Item SID: <|sid_begin|><s_a_7><s_b_112><s_c_18><s_d_204><|sid_end|>
Title: Hair One Cleanser and Conditioner with Argan Oil for Curly Hair 12 oz
Categories: Beauty > Hair Care > Shampoos

 # Candidate Items

Candidate 1: One 'n Only Argan Oil Styling Cream, 10 fl. oz.
Categories: Beauty > Hair Care > Styling Products > Creams, Gels & Lotions

Candidate 2: One 'n Only Argan Oil Spray Treatment 6 fl. oz
Categories: Beauty > Hair Care > Hair & Scalp Treatments

Candidate 3: Deva Devacurl One Condition Conditioner, 12 Ounce
Categories: Beauty > Hair Care > Conditioners

Candidate 4: Giovanni Hair Care - Direct Leave-In Conditioner, 8.5 fl oz
Categories: Beauty > Hair Care > Hair Relaxers > Conditioners

Candidate 5: Paul Mitchell The Conditioner, Leave-in Moisturizer, 10.14-ounce
Categories: Beauty > Hair Care > Conditioners

Candidate 6: Kinky-Curly Knot Today Leave In Conditioner/Detangler - 8 oz
Categories: Beauty > Hair Care > Conditioners

Candidate 7: Hair One Cleanser and Conditioner with Olive Oil for Dry Hair 12 oz

Categories: Beauty > Hair Care > Hair Loss Products > Conditioners

Candidate 8: It's A 10 Miracle Leave In Product, 4-Ounces
Categories: Beauty > Hair Care > Conditioners

Candidate 9: It's A 10 Miracle Moisture Shampoo, 10-Ounce Bottle
Categories: Beauty > Hair Care > Shampoos

Candidate 10: Dabur Vatika Enriched Coconut Hair Oil 150ml (Pack of 2)
Categories: Beauty > Hair Care > Hair & Scalp Treatments

 # Task

Analyze the user's purchase history and predict which candidate they are most likely to purchase next. Think step-by-step about the patterns and preferences shown in their history.

Critical Guidelines:

1. **Cite items by SID**: When referring to items in the purchase history, cite them directly using their SID (e.g., <|sid_begin|><s_a_99><s_b_19><s_c_220><s_d_204><|sid_end|>). This allows the model to learn to reason with semantic IDs naturally interleaved with language.
2. Focus on analyzing patterns in the user's purchase history
3. Think about sequential purchase patterns (e.g., shampoo → conditioner, phone → case)

 # Example Output Format

Step 1: "Looking at the user's purchase history, <|sid_begin|>...<|sid_end|> and <|sid_begin|>...<|sid_end|> are both hair care products, indicating a focus on hair maintenance."

Step 2: "The recent purchase of <|sid_begin|>...<|sid_end|> (a deep conditioning treatment) suggests the user is looking for products that provide intensive hair care."

Step 3: "Based on this pattern, Candidate 3 would be the natural next purchase as it complements their existing routine."
Prediction: Candidate 3

 # Your Response Format

Step 1: "<analyze the user's purchase patterns, citing specific items by their SID>"
Step 2: "<identify what type of product would complement their recent purchases, citing items by SID>"
Step 3: "<explain which candidate best matches the user's needs>"
Prediction: Candidate <number>

Reasoning:

Generated Reasoning Trace (Model Output)

{
"explanation": "

Step 1: "Looking at the user's purchase history, <|sid_begin|><s_a_57><s_b_7><s_c_213><s_d_26><|sid_end|> and <|sid_begin|><s_a_7><s_b_112><s_c_18><s_d_204><|sid_end|> are both hair care products, specifically for curly hair, indicating a focus on maintaining curly hair with argan oil."

Step 2: "The purchase of a cleanser and conditioner (<|sid_begin|><s_a_7><s_b_112><s_c_18><s_d_204><|sid_end|>) suggests the user follows a routine of shampoo and conditioner, which is typically followed by styling or treatment products."

Step 3: "Candidate 1 (One 'n Only Argan Oil Styling Cream) and Candidate 2 (One 'n Only Argan Oil Spray Treatment) both align with the brand and product type the user has previously purchased. However, since styling products are typically used after cleansing and conditioning, Candidate 1 is the most logical next step in the user's hair care routine."

Based on the user's purchase history and category patterns, Candidate 1 best matches the predicted category and complements their recent purchases.",

"recommendations": ["1", "2", "3", "4", "5", "6", "7", "8", "9", "10"]

```
}
```

Color Legend: Section Headers | Semantic IDs (SID) | Predictions/Answers | Guidelines | Placeholders

E Methodology Details

Robust Parsing of Reasoning-aware Re-ranking Output

Input: raw LLM output O, number of candidates $|D|$
Output: reasoning trace O_R (optional), validated ranking O_D (or `None`)

Algorithm: Robust Re-ranking Output Parsing

1. **Initialize** $O_R \leftarrow$ `None`, $O_D \leftarrow$ `None`
2. **Stage I: JSON-based Structured Parsing**
   (a) Extract all JSON-like substrings from O
   (b) Traverse substrings in reverse order
   (c) For each substring:
   - Attempt JSON parsing
   - If successful:
     – Extract reasoning field (`explanation` or `reasoning`)
     – Extract ranking field (`recommendations` or `ranking`)
     – If ranking is a non-empty list, break

   ▷ *Prefer well-structured outputs while tolerating extra text*
3. **Stage II: Regex-based Fallback Parsing (if Stage I fails)**
   (a) Search for ranking patterns using regular expressions
   (b) Extract all numeric indices from the matched span
   (c) Optionally extract reasoning text if present

   ▷ *Handles malformed or partially structured LLM generations*
4. **Stage III: Ranking Validation and Canonicalization**
   (a) Filter ranking entries to valid integers in $[1, |D|]$
   (b) Remove duplicates while preserving order
   (c) If no valid index remains, return (`None`, `None`)
   (d) Convert to zero-based indices
   (e) Append missing candidates to form a full permutation of length $|D|$

   ▷ *Guarantees a valid permutation for reward computation*

Return: (O_R, O_D) if a valid ranking is recovered; otherwise (`None`, `None`)

F Case Study

GR2's Case Study for Targeted Reasoning with Knowledge Priming

 # System Role

You are an expert at analyzing e-commerce purchase patterns and predicting user preferences. Given the user's purchase history (with SID identifiers) and a list of candidate items, predict which item the user is most likely to purchase next by re-ranking the pre-ranked candidates.

 # User Purchase History

Item 1
Item SID: <|sid_begin|><s_a_134><s_b_194><s_c_131><s_d_121><|sid_end|>
Title: Pureology Hydrate Shampoo 33.8 oz & Condition 33.8 oz Duo Set
Categories: Beauty > Hair Care > Shampoo & Conditioner Sets

Item 2
Item SID: <|sid_begin|><s_a_120><s_b_95><s_c_240><s_d_161><|sid_end|>
Title: Pureology Hydrate Shampoo, 8.5 Ounce
Categories: Beauty > Hair Care > Shampoos

Item 3
Item SID: <|sid_begin|><s_a_238><s_b_79><s_c_36><s_d_141><|sid_end|>
Title: Pureology Anti-Fade Complex Hydrate Conditioner, 8.5 Ounce
Categories: Beauty > Hair Care > Conditioners

 # Candidate Items

Candidate 1
Item SID: <|sid_begin|><s_a_238><s_b_194><s_c_120><s_d_24><|sid_end|>
Title: Pravana Pure Light Sulfate-free Brightening Shampoo
Categories: Beauty > Hair Care > Shampoos

Candidate 2
Item SID: <|sid_begin|><s_a_134><s_b_194><s_c_59><s_d_74><|sid_end|>
Title: Pureology Hydrate Shampoo 8.5oz and Hydrate Conditioner 8.5oz Duo
Categories: Beauty > Hair Care > Shampoo & Conditioner Sets

Candidate 3
Item SID: <|sid_begin|><s_a_155><s_b_123><s_c_248><s_d_251><|sid_end|>
Title: L'Oreal Paris EverSleek Sulfate-Free Smoothing Shampoo
Categories: Beauty > Hair Care > Shampoos

Candidate 4
Item SID: <|sid_begin|><s_a_137><s_b_63><s_c_202><s_d_132><|sid_end|>
Title: Revlon RV544PKF Ionic Ceramic Hair Dryer
Categories: Beauty > Hair Care > Styling Tools > Hair Dryers

Candidate 5
Item SID: <|sid_begin|><s_a_94><s_b_56><s_c_151><s_d_226><|sid_end|>
Title: Cetaphil Moisturizing Cream (Pack of 3)
Categories: Beauty > Skin Care > Body > Moisturizers

Candidate 6
Item SID: <|sid_begin|><s_a_238><s_b_178><s_c_125><s_d_87><|sid_end|>
Title: Clairol Shimmer Lights Shampoo Blonde & Silver 31.5oz
Categories: Beauty > Hair Care > Shampoos

Candidate 7
Item SID: <|sid_begin|><s_a_238><s_b_56><s_c_125><s_d_9><|sid_end|>
Title: Clairol Professional Shimmer Lights Shampoo 31.5oz
Categories: Beauty > Hair Care > Shampoos

Candidate 8
Item SID: <|sid_begin|><s_a_134><s_b_209><s_c_245><s_d_42><|sid_end|>
Title: Agadir Argan Oil Shampoo + Conditioner Combo Set
Categories: Beauty > Hair Care > Shampoo & Conditioner Sets

Candidate 9
Item SID: <|sid_begin|><s_a_243><s_b_182><s_c_89><s_d_5><|sid_end|>
Title: Cricket Technique #450 Thermal Brush
Categories: Beauty > Hair Care > Styling Tools > Brushes

Candidate 10
Item SID: <|sid_begin|><s_a_155><s_b_202><s_c_218><s_d_11><|sid_end|>
Title: Fekkai Glossing Hair Products Starter Kit
Categories: Beauty > Hair Care

 # Model Output

Reasoning Trace

Step 1 Reasoning: "The user has a history of purchasing hair care products, specifically shampoo and conditioner sets. is a Pureology Hydrate Shampoo and Conditioner Duo Set, and and are individual Pureology Hydrate Shampoo and Conditioner products. This indicates a consistent interest in hair care, particularly in hydrating products."

Step 1 Category: "Beauty > Hair Care"

Step 2 Reasoning: "The user has specifically purchased hydrating shampoo and conditioner sets, which suggests a focus on maintaining hair health and moisture. The Pureology Hydrate line is known for its hydrating properties, and the user has bought both the shampoo and conditioner, indicating a preference for complete sets. This pattern aligns with the candidate items that are also shampoo and conditioner sets, such as <|sid_begin|><s_a_134><s_b_194><s_c_59><s_d_74><|sid_end|> (Candidate 2) and <|sid_begin|><s_a_134><s_b_209><s_c_245><s_d_42><|sid_end|> (Candidate 8)."

Based on the user's purchase history and category patterns, Candidate 2 best matches the predicted category and complements their recent purchases.

Re-ranked Candidates

[2, 8, 1, 3, 4, 5, 6, 7, 9, 10]

 # Ground Truth

<|sid_begin|><s_a_134><s_b_194><s_c_59><s_d_74><|sid_end|> (Candidate 2)

G Evaluation Metrics

Recall@K. Let $\mathcal{U}$ denote the set of users, i_u^* the ground-truth next item for user $u \in \mathcal{U}$, and $\mathcal{R}_u^K$ the set of top-K items ranked by the model. Recall@K is defined as

$$\text{Recall@}K = \mathbb{E}_{u \in \mathcal{U}} \left[\mathbb{I}\big(i_u^* \in \mathcal{R}_u^K\big) \right], \tag{27}$$

where $\mathbb{I}(\cdot)$ is the indicator function.

NDCG@K. For user u, the Discounted Cumulative Gain at K (DCG@K) is computed as

$$\text{DCG@}K = \sum_{j=1}^{K} \frac{2^{\text{rel}_{u,j}} - 1}{\log_2(j+1)}, \tag{28}$$

where $\text{rel}_{u,j} \in \{0, 1\}$ indicates whether the item ranked at position j matches the ground-truth next item i_u^*.

The Ideal DCG at K (IDCG@K) is obtained by placing the ground-truth item at the first position, in the standard leave-one-out evaluation protocol for sequential recommendation which is always 1: $\text{IDCG@}K = \frac{1}{\log_2(1+1)} = 1$. NDCG@K is then defined as

$$\text{NDCG@}K = \mathbb{E}_{u \in \mathcal{U}} \left[\frac{\text{DCG@}K}{\text{IDCG@}K} \right]. \tag{29}$$

SID Uniqueness/Collision Rate To evaluate the quality of the learned SIDs, we measure the *collision rate*, which quantifies how often multiple items are assigned the same SID. A lower collision rate indicates stronger discriminative power of the tokenizer and a more faithful semantic encoding of items.

Let $\mathcal{I}$ denote the set of items, and let $\text{SID}(i)$ be the SID assigned to item $i \in \mathcal{I}$. We define the collision set as

$$\mathcal{C} = \{ i \in \mathcal{I} \mid \exists j \in \mathcal{I},\ j \neq i,\ \text{SID}(i) = \text{SID}(j) \}. \tag{30}$$

The *collision rate* is then computed as

$$\text{CollisionRate} = |\mathcal{C}|/|\mathcal{I}|. \tag{31}$$

and the SID uniqueness is computed as

$$\text{Uniqueness} = 1 - \text{CollisionRate} \tag{32}$$